

How to compile with Emscripten (emcc)

Emscripten is used to compile a project and turn source files, raylib source modules and audio assets into WebAssembly binaries. For my project, I worked alongside Gemini to create a command for Command Prompt that would use emcc to build my project. The command used is as follows:

```
"emcc -o web/index.html Space_Invaders.cpp C:/raylib/raylib/src/rcore.c C:/raylib/raylib/src/rshapes.c  
C:/raylib/raylib/src/rtextures.c C:/raylib/raylib/src/rtext.c C:/raylib/raylib/src/rmodels.c  
C:/raylib/raylib/src/raudio.c --preload-file Laser.wav --preload-file Explosion.wav --preload-file GameOver.wav  
--preload-file BackgroundMusic.wav -Os -Wall -I C:/raylib/raylib/src -s USE_GLFW=3 -s ASYNCIFY -  
DPLATFORM_WEB -DGRAPHICS_API_OPENGL_ES2 --shell-file C:/raylib/raylib/src/shell.html"
```

After making sure this works, I then looked into what each part of this command actually does, which I have listed below.

- `-o web/index.html`: Tells emcc to output index.html along with generating the matching index.js, index.wasm and index.data.
- `rcore.c, rshapes.c, rtextures.c, rtext.c, rmodels.c, raudio.c`: Passes the main Space_Invaders.cpp script along with Raylib's core implementation files.
- `--preload-file`: packages the audio files directly into a virtual filesystem stored inside index.data so browser applications can read them at runtime.
- `-Os -Wall`: `-Os` shrinks the WebAssembly binary size for web delivery, while `-Wall` displays compiler warnings.
- `-I C:/raylib/raylib/src`: Adds Raylib's header files (raylib.h) to emcc's search path
- `-s USE_GLFW=3 -s ASYNCIFY`: Enables GLFW support for web browser input/window handling and allows main loop execution without freezing browser UI threads.
- `-DPLATFORM_WEB -DGRAPHICS_API_OPENGL_ES2`: Instructs Raylib to compile using web architecture and forces WebGL 1.0 shader compatibility.
- `--shell-file C:/raylib/raylib/src/shell.html`: Embeds the compiled canvas inside Raylib's web page wrapper.

How to serve and play in a browser (e.g., `python3 -m http.server 8000`)

To be able to play this game in your browser, it needs to be locally hosted using Command Prompt and Python. To do this, you need to locate the folder containing all of the game's files (.data, .html, .js and .wasm) and open the terminal inside the folder. Once the terminal is open, input the following command "`python3 -m http.server 8000`". This command uses Python to host your game locally in your browser. To be able to access and play your game, you now need to use the following link in the browser of your choosing "`http://localhost:8000/`". This link will take you to the location where your game is being hosted on. The server number can be changed if you would like in the command to any other number but then the link will have to change to reflect the number in the Command Prompt.

The source/API you used for the web request

For this task, I had my game access the weather report from `api.open-meteo.com`. Using the current weather, my game would access current wind speeds to affect the bullets shot by the player. This part of the development took the longest of all features created as implementing the API took a lot of code in the first place but then when it came to testing, I kept finding out that the wind was too strong and the bullets were disappearing off screen. To fix this issue, I had to adjust many variables until finally the wind was affecting the bullets direction whilst also having it still be possible to kill the enemies.

My original idea was to make an online scoreboard but I ended up realising this idea was possible and instead switched to the wind web request idea.